Create an Environment in Postman

Setting up a Reusable Environment with Variables

Hands-On	11
Scenario	Create an Environment in Postman
Tool	Postman (Desktop or Web)
Duration	20 minutes
Prerequisite	Workspace and Collection already created

Objective

The objective of this assignment is to create an **Environment** inside Postman. An environment is a named set of **variables** (such as base URLs, usernames, passwords, tokens, and resource ids) that can be referenced from any request in the workspace using the `{{variable_name}}` syntax. Environments make it easy to switch between different setups (for example: *Development*, *Staging*, *Production*) without editing each request individually.

Why Use Environments?

Without environments, every API request would have to hard-code its URL, credentials, and resource ids. This makes the collection brittle and difficult to maintain. With environments:

- **Avoid duplication —** set `{{base_url}}` once and reference it across every request.
- **Switch contexts instantly —** swap between Dev, Staging and Production by changing the active environment in the top-right selector.
- **Keep secrets safer —** mark variables as **secret** so passwords and tokens are masked in the UI and exports.
- **Chain requests dynamically —** tests can write back into environment variables (e.g. capture a `token` from /auth and reuse it in later requests).

Steps to Create an Environment

1

Go to the Right Workspace

Open Postman (desktop or web) and sign in if not already logged in.

Click the **Workspaces** dropdown and select the workspace where the environment should live (typically the same workspace that holds the related collection).

2 Open the Environments Tab and Click +

In the left sidebar of the workspace, click the **Environments** tab.

Click the **+** (Create new) button at the top of the Environments panel.

A new untitled environment opens in the central editor.

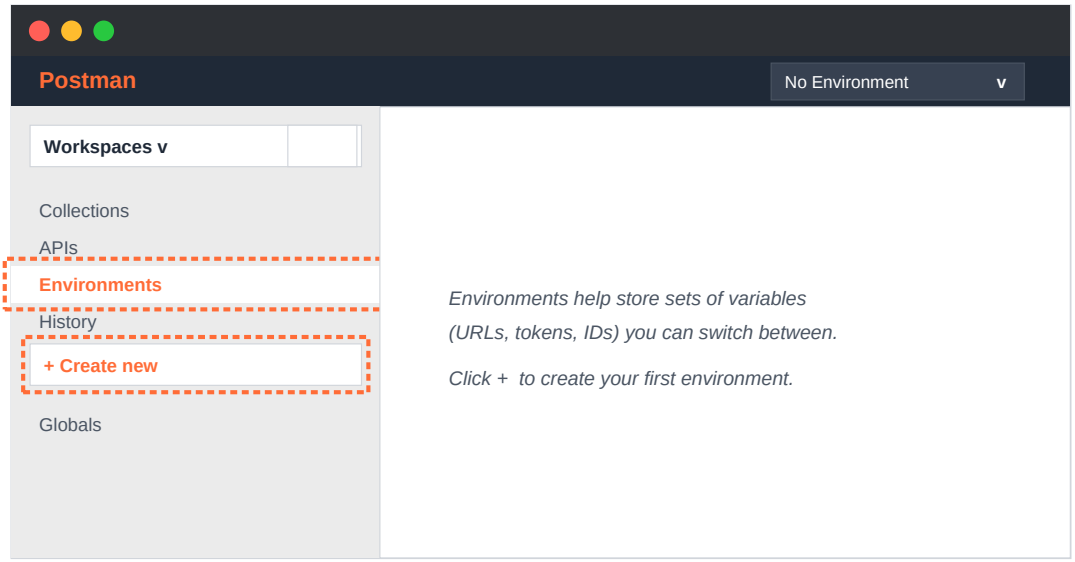

*Figure 1 — Left sidebar showing the **Environments** tab selected, with the **+ Create new** button highlighted.*

3 Enter a Name for the New Environment

At the top of the environment editor, click the title field and type a clear, descriptive name — for example *Restful Booker Env*, *Bitcoin API - Dev*, or simply *Staging*.

A meaningful name makes it easy to identify the right environment in the dropdown selector later.

4 Add the Required Variables

In the variables table below the name field, fill in one row per variable. Each row has four columns:

- **Variable** — the name (no spaces, e.g. `base_url`).
- **Type** — *default* for normal values, *secret* for tokens and passwords (Postman masks these).
- **Initial value** — the value synced to the cloud and shared with the team.
- **Current value** — the value used locally; can be different from the initial value and is not synced.

Variables can be added now or later — the environment can be saved empty and updated whenever new variables are needed.

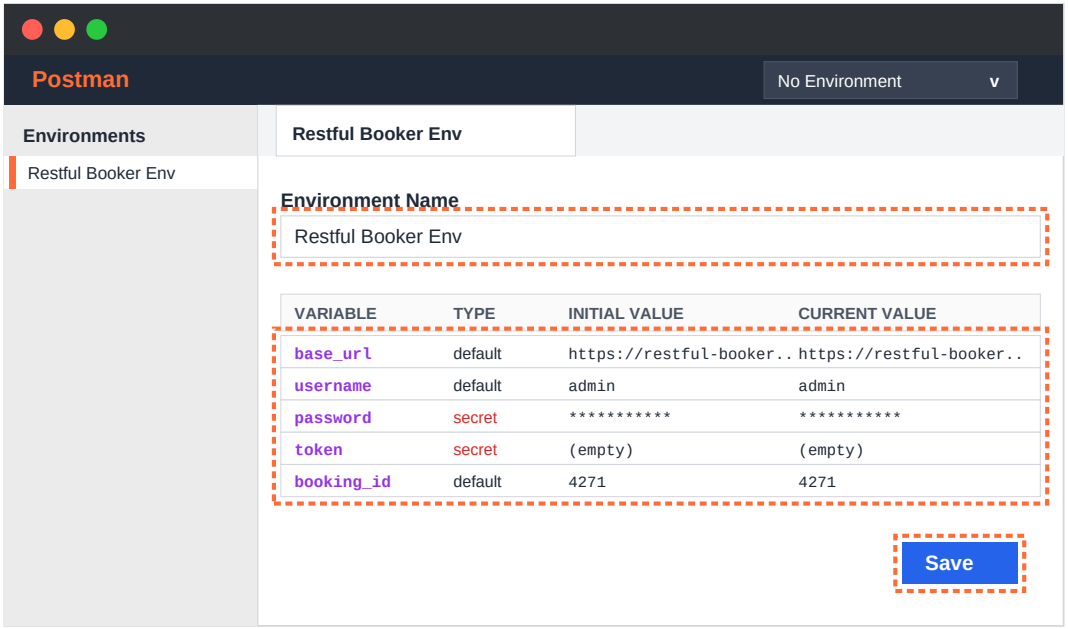

*Figure 2 — Environment editor with the name **Restful Booker Env** and a typical set of variables: base_url, username, password (secret), token (secret), and booking_id.*

Sample Variables (Suggested for Restful Booker)

If no specific variables are required, use the following common set as a starting point. Each variable can later be referenced inside any request as `{{variable_name}}`:

Variable	Type	Initial Value
`base_url`	default	https://restful-booker.herokuapp.com
`username`	default	admin
`password`	secret	password123
`token`	secret	(populated by /auth response)
`booking_id`	default	(populated by POST /booking response)

5

Save the Environment Variables

Click the **Save** icon (or press **Ctrl + S** / **Cmd + S**) to persist the environment and all its variables.

Once saved, the environment becomes available in the environment selector in the top-right corner of the workbench.

Any further changes (adding, editing, removing variables) need to be saved again to take effect.

6 Activate the Environment

To start using the new environment, click the environment selector in the top-right corner of the workbench (it shows *No Environment* by default).

From the dropdown, choose the environment that was just created (e.g. *Restful Booker Env*).

Postman now resolves any `{{variable_name}}` references in URLs, headers, and bodies against that environment.

Example: a request URL of `{{base_url}}/booking/{{booking_id}}` will automatically expand to `https://restful-booker.herokuapp.com/booking/4271`.

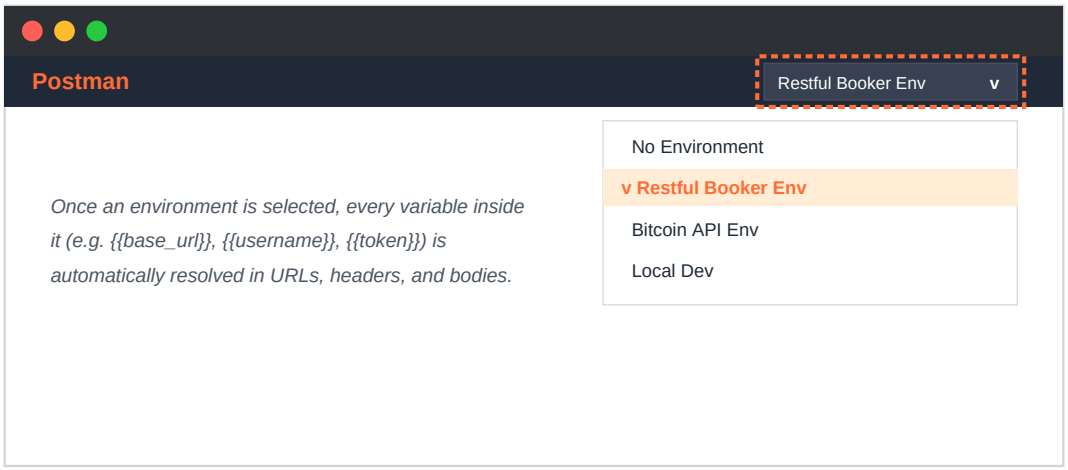

*Figure 3 — The **environment selector** at the top-right of the workbench, with the new **Restful Booker Env** selected as the active environment.*

Using Variables in Requests

Once the environment is active, variables can be referenced in URLs, headers, query params, request bodies, and even test scripts. The syntax is double curly braces around the variable name. Examples:

```
URL:      {{base_url}}/booking/{{booking_id}}
Header:  Cookie: token={{token}}
Body:    { "username": "{{username}}", "password": "{{password}}" }
Script:  pm.environment.set("token", pm.response.json().token);
```

Verification Checklist

1. The new environment appears in the **Environments** tab in the left sidebar.
2. Opening the environment shows the variables table with all the entered rows.
3. The environment is selectable from the dropdown in the top-right of the workbench.
4. After activating it, any `{{variable}}` reference in a request resolves to the expected value (visible by hovering over the variable name in the URL bar).
5. Variables marked as **secret** appear as masked dots in the editor.

Conclusion

A reusable Postman **Environment** has been successfully created. All future requests in the workspace can now reference its variables with the `{{variable_name}}` syntax instead of hard-coding URLs, credentials, and ids. The environment will be particularly useful in upcoming assignments that introduce test scripts — tokens captured from one request can be saved into the environment and automatically picked up by later requests.